

Conditional statements and Boolean expressions

Introduction to Programming in Python

Rae Harbird, r.harbird@ucl.ac.uk

Conditions and boolean expressions

Using if and else

Relational operators

Comparing floating point numbers

Nested if-else

Boolean operators

Our example program

Conditions and boolean expressions

Software development

Software development consists of:

- ▶ Establishing the **requirements**
- ▶ Creating a **design**
- ▶ **Implementing** the code, and
- ▶ **Testing** the implementation.

Software development

Software **requirements** specify *what* a program must accomplish.

Software **design** specifies *how* the requirements will be accomplished. Software design involves the design of **algorithms**.

An algorithm is a step-by-step process for doing a task, often first written in **pseudocode**.

Good programmers always leave plenty of time for testing. Testing should be *methodical*, in order to maximise the chance of finding errors.

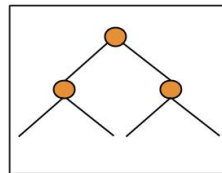
Flow of Control



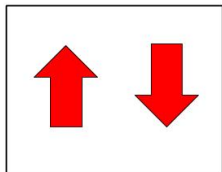
Sequence



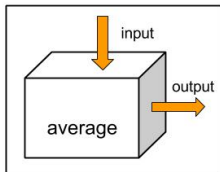
Loop



Selection



Input / Output



Functions

Figure 1: Control structures

Flow of Control

The **flow of control** is the order that statements are executed in a program.

Unless otherwise specified, Python programs start with the first statement in the file and finish with the last (shown as sequence in the diagram).

This behaviour can be modified with selection statements, also called conditional statements, loops or repetition statements, input and output statements and functions. Any algorithm can be expressed in terms of these control structures.

Conditions

Conditional statements allow the program to **select** which statement will be executed next, based on the truth or falsity of a given **condition**.

The conditional statements in Python are the `if` statement, the `if-else` and the `if-elif-else` statements.

A Random Number Example

We will use variations of the following program in order to illustrate the use of conditional statements:

```
# Name: generate_random.py
```

```
# Description: This program generates a random number.
```

```
import random
```

```
MIN = 1
```

```
MAX = 5
```

```
def main():
```

```
    randomNumber = random.randint(MIN, MAX)
```

```
    print("A random number between " + str(MIN), end="")
```

```
    print(" and " + str(MAX) + ": " + str(randomNumber))
```

```
if __name__ == "__main__" :
```

```
    main()
```

Explanation

The program generates (and in this first variation outputs) a random number between MIN and MAX (including MAX). For example, the output might be:

A random number between 1 and 5: 4

The random module

The program `guess_one_random.py` includes the statement:
`import random`. The `import` statement allows us to make use of code that other people have written, code is organised as libraries.

Take a look at the libraries available for your version of Python. Python's standard libraries include a module named **random** that contains many useful functions related to generating random values.

Technically the numbers generated by the library are called *pseudo-random numbers* because they are generated from mathematical functions and the system clock.

A call of `random.randint(1, 10)` returns a random integer from 1 through 10 inclusive.

Using if and else

Guessing With if Statements

The program `guess_one_random.py` uses two `if` statements to output two alternative messages. Each `if` statement has its own condition.

```
import random
```

```
MIN = 1
```

```
MAX = 5
```

```
def main():
```

```
    answer = random.randint(MIN, MAX)
```

```
    guess = int(input("\n\tGuess the number between " + str(MIN)  
                    + " and " + str(MAX) + ": "))
```

```
    if guess == answer:
```

```
        print("\tCorrect - well done!")
```

```
    if guess != answer:
```

```
        print("\tNo, the answer was " + str(answer) + ".\n")
```

Note: the two conditions are exact “opposites”.

Guessing with an if-else

This code snippet from our program shows how the program could be modified to use a single if-else statement:

```
answer = random.randint(MIN, MAX)

guess = int(input("\\n\\tGuess the number between " + str(MIN)
                  + " and " + str(MAX) + ": "))

if guess == answer:
    print("\\tCorrect - well done!")
else:
    print("\\tNo, the answer was " + str(answer) + ".\\n")
```

Try it out. Note: there is no separate condition explicitly included in the “else” part.

Blocks and indentation

Python code is organised as blocks. We must indent all the statements to be executed if a condition is `True`, likewise, we must indent all the statements to be executed if the condition is `False`.

```
if totalSales > 100.0 :
    ↑ discount = totalSales * 0.05
    | totalSales = totalSales - discount
    | print("You received a discount of $%.2f" % discount)
else :
    ↑ diff = 100.0 - totalSales
    | if diff < 10.0 :
    |     ↑ print("If you were to purchase our item of the day you can receive a 5% discount.")
    |     else :
    |         ↑ print("You need to spend $%.2f more to receive a 5% discount." % diff)
    |         |
    |         |
    |         |
0  1  2  Indentation level
```

Figure 2: Indentation

When editing code, it's best to use the tab key to indent your code. Your editor will probably allow you to choose the size of the tab, set it to 4 spaces. Don't mix spaces and tabs in the same program.

Using a `main()` function

It's good to structure your programs using a `main()` function. At the moment we won't go into the reasons for this. We'll explain the `main` function in more detail when we cover functions.

If-else statements in general

The general form of `if-else` statements is:

```
if condition :  
    statement(s)  
else :  
    alternative statement(s)
```

If the condition evaluates to `True` then the indented statements(s) are executed, whereas if it is `False` the “alternative statement(s)” get executed.

The whole of the `else` part is optional.

Relational operators

Relational operators

Every if statement contains a condition. In many cases, the condition involves comparing two values. In Python, you use a *relational operator* to do this.

Python	Math Notation	Description
>	>	Greater than
>=	≥	Greater than or equal to
<	<	Less than
<=	≤	Less then or equal to
==	=	Equal
!=	≠	Not equal

Precedence and Relational Operators

Relational operators have a lower precedence than arithmetic operators.

That means you can write arithmetic expressions on either side of the relational operator without using parentheses.

For example, in the expression:

```
floor - 1 < 13
```

both sides (floor - 1 and 13) of the < operator are evaluated, and the results are compared.

Comparing strings

- ▶ Strings can also be compared using Python's relational operators. For example, to test whether two strings are equal, use the `==` operator:

```
if name1 == name2 :  
    print("The strings are identical.")
```

- ▶ or to test if they are not equal, use the `!=` operator:

```
if name1 != name2 :  
    print("The strings are not identical.")
```

- ▶ For two strings to be equal, they must be of the same length and contain the same sequence of characters. If even one character is different, the two strings will not be equal.

Strings and lexicographical ordering

- ▶ Python uses *lexicographical ordering* to compare strings.

```
if "cat" > "bat":  
    print("bat comes before cat")  
else:  
    print("cat comes before bat")
```

Output:

```
bat comes before cat
```

Test yourself

1. Give the opposite of the condition `floor > 13`
2. A programmer wrote the condition `floor = 13` to test whether the floor is 13. Fix the error.
3. Supply a condition in this `if` statement to test whether the user entered a Y:

```
response = input("Enter Y to quit.")  
if . . . :  
    print("Goodbye")
```

4. How do you test that a string variable: `response` is the empty string?

Comparing floating point numbers

Common Error - Comparing floating point numbers

Floating-point numbers have only a limited precision, and calculations can introduce rounding errors.

For example, the following code multiplies the square root of 2 by itself. Ideally, we expect to get the answer 2:

```
from math import sqrt

r = sqrt(2.0)
if r * r == 2.0 :
    print("sqrt(2.0) squared is 2.0")
else :
    print("sqrt(2.0) squared is not 2.0 but", r * r)
```

Output:

```
sqrt(2.0) squared is not 2.0 but 2.0000000000000004
```

Solution - Comparing floating point numbers

To get around this, test whether the numbers are close enough. Mathematically, we would write:

$$|x - y| < \epsilon$$

Means that x and y are close enough if the difference between them is a very small number, ϵ .

It is common to set ϵ to 10^{-14} when comparing floating-point numbers:

```
from math import sqrt

EPSILON = 1E-14
r = sqrt(2.0)
if abs(r * r - 2.0) < EPSILON :
    print("sqrt(2.0) squared is approximately 2.0")
```

Nested if-else

Nested if-else

if-else statements can be strung together as follows:

```
if boolean exp 1 :  
    statement(s) 1  
elif boolean exp 2 :  
    statement(s) 2  
elif boolean exp 3 :  
    statement(s) 3  
else :  
    statement(s) 4
```

Python executes the first set of statements for which the corresponding boolean expression is true, and ignores the rest.

Why?

For example, if boolean exp 1 is false and both boolean exp 2 and boolean exp 3 are true, only statement(s) 2 will be executed.

```
if boolean exp 1 :  
    statement(s) 1  
else :  
    if boolean exp 2 :  
        statement(s) 2  
    else :  
        if boolean exp 3 :  
            statement(s) 3  
        else :  
            statement(s) 4
```

Guessing Two Numbers In Order

See program `guess_two_randoms.py` in the folder: `src`.

It's not perfect, we have to guess the numbers in a specific sequence. How can we improve this?

Boolean operators

Boolean Operators

George Boole developed a branch of mathematics, Boolean algebra, which was fundamental to the dawn of the Information Age.

Boolean values can be either True or False.

We can use the **boolean operators**, also called logical operators, and (AND), or (OR) and not (NOT) to generate new boolean values from existing ones.

Example	Value
<code>(1 == 2) or (8 == 4+4)</code>	True
<code>(1 == 2) and (8 == 4+4)</code>	False
<code>((1<2) or (8 == 4+4)) and (3>4)</code>	False
<code>(1<2) or ((8 == 4+4) and (3>4))</code>	True
<code>not(1<2) or ((8 == 4+4) and (3>4))</code>	False

Logical operators

Operator	Meaning	Example	Value
and	conjunction	(2 == 2) and (3 < 4)	True
or	conjunction	(1 < 2) or (2 == 3)	True
not	negation	not (2 == 2)	True

Logical operators - Truth table for `not`

p	not p
True	False
False	True

Logical operators - Truth table for `and` and `or`

p	q	p and q	p or q
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

Our example program

Guessing two Numbers simplified

Take a look at the program “guess_two_randoms_simplified.py”.

Here is an extract:

```
first_answer = random.randint(MIN, MAX)
second_answer = random.randint(MIN, MAX)

first_guess = int(input("\n\tGuess the first number between "
                        + str(MIN) + " and " + str(MAX) + ": "))
second_guess = int(input("\n\tGuess the second number between "
                        + str(MIN) + " and " + str(MAX) + ": "))

if first_guess == first_answer:
    if second_guess == second_answer:
        print("\tCorrect - well done!")
    else:
        print("\tNo, the answers were ", end='')
        print(str(first_answer) + " and "
              + str(second_answer) + ".\n")
else:
    print("\tNo, the answers were ", end='')
    print(str(first_answer) + " and " + str(second_answer) + ".\n")
```

Guessing in either order

Now take a look at the program `guess_two_randoms_either_order.py`.

```
first_answer = random.randint(MIN, MAX)
second_answer = random.randint(MIN, MAX)

first_guess = int(input("\n\tGuess the first number between "
                        + str(MIN) + " and " + str(MAX) + ": "))
second_guess = int(input("\n\tGuess the second number between "
                        + str(MIN) + " and " + str(MAX) + ": "))

if ((first_guess == first_answer and
    second_guess == second_answer) or

    (first_guess == second_answer and
    second_guess == first_answer)):

    print("\tCorrect - well done!")
else:
    print("\tNo, the answers were ", end='')
    print(str(first_answer) + " and " + str(second_answer) + ".\n")
```

Guessing in Either Order Improved

Improved again in guess_two_randoms_boolean.py.

```
first_answer = random.randint(MIN, MAX)
second_answer = random.randint(MIN, MAX)

first_guess = int(input("\n\tGuess the first number between "
                        + str(MIN) + " and " + str(MAX) + ": "))
second_guess = int(input("\n\tGuess the second number between "
                        + str(MIN) + " and " + str(MAX) + ": "))

okSameOrder = first_guess == first_answer
                and second_guess == second_answer
okOtherOrder = first_guess == second_answer
                and second_guess == first_answer

if okSameOrder or okOtherOrder:
    print("\tCorrect - well done!")
else:
    print("\tNo, the answers were ", end='')
    print(str(first_answer) + " and "
          + str(second_answer) + ".\n")
```

Test yourself

Expression	Value	Comment
3 <= 4	True	3 is less than 4.
3 =< 4	Error	The “less than or equal” operator is <=, not =<.
3 > 4	False	> is the opposite of <=.
4 > 4	False	
4 <= 4	True	Both sides are equal.
3 == 5 - 2	True	== tests for equality.
3 != 5 - 1	True	!= tests for inequality. It is true that 3 is not 5 - 1.
3 = 6 / 2	Error	Use == to test for equality.
1.0 / 3.0 == 0.3333333333	False	A fractional value will not have this level of accuracy.
"10" > 5	Error	You cannot compare a string to a number.